

Delivering models to LUCKBOUND

How to hand over art so it drops straight into the game — and why we do not build directly in the shared place.

The one-sentence version

Send us **.rbxmx prefabs** (or uploaded mesh asset IDs). Do **not** build in the live place, and do **not** edit scripts in Studio.

Save as .rbxmx, not .rbxm — the x matters

Both are Roblox model files and both work in the game. The difference is that **.rbxm is binary and .rbxmx is XML**.

The programmer's tooling can **read an .rbxmx directly** — every part name, attachment and pivot — and wire the animation code against what is actually in the file. A binary .rbxm is opaque to it, so the wiring has to be written blind against a name list, and a single typo only shows up at runtime.

In Studio: **Save to File...** and pick **Roblox XML model files (*.rbxmx)** in the file-type dropdown. That is the only change.

Why not just build in the place?

The game's code lives in a git repository, not in the Roblox place file. A tool called **Rojo** continuously copies code from the repo into Studio. That flow is **one-directional**: repo → Studio.

Two consequences, and they are the whole reason for this document:

- **Anything you change in Studio never reaches the repo.** There is no sync back. It exists only in your copy until someone saves the place, and it is invisible to version control.
- **Scripts you edit in Studio get overwritten.** Next time Rojo syncs, the repo's version wins and your edit is gone with no warning.

Geometry is different — Rojo does not manage Workspace, so models you place there survive. But they still are not in version control, so they cannot be reviewed, rolled back, or rebuilt by anyone else. Hence: hand them over as files.

What to send

Format	Use it for	Why
.rbxmx (XML model file)	Anything with moving parts, lights, particles, or attachment points — the Fate Engine, the Expedition Gate, portals.	Keeps the whole hierarchy: separate parts, lights, emitters, attachments and attribute values. This is what lets us animate it — and being XML, we can read it to confirm the names before writing any code.
Mesh asset ID (rbxassetid://...)	Single rigid objects — terrain chunks, props, scenery, an island.	Just geometry. Cheapest to deliver and to load. No moving sub-parts.

Format	Use it for	Why
.blend / .fbx	Source only. Keep it in the repo for traceability.	Roblox cannot load these at runtime. They must be uploaded and referenced by asset ID first. There is no way around this.

The rule that decides which one

Does any part of it need to move independently? Then it is an **.rbxmx**, with each moving piece as its own part.

A single welded mesh can be spun, tinted or faded *as a whole*. It can never have an inner ring that rotates against an outer ring — they are one rigid object.

Where files go

Everything lives in the repository, in folders that already exist:

```
assets/
  source/    your .blend files      (authoring only, never loaded by the game)
    worlds//          per-biome pieces
  props/    characters/ discoveries/
  export/   your .fbx exports      (mirror the same paths as source/)
  rbxm/     finished .rbxmx prefabs <-- deliver here
```

Mirror the paths. If the source is `assets/source/props/fate_engine.blend`, the export is `assets/export/props/fate_engine.fbx`.

Export settings that matter

These six are the ones that cost an evening when they are wrong.

Setting	Value	If you get it wrong
Scale	1 Blender metre = 1 Roblox stud	Everything arrives 10× or 0.1×. Check against the R6 rig (below).
Up axis	Y up, -Z forward	Blender is Z-up, Roblox is Y-up. Model arrives lying on its side.
Apply Transform	On	Rotation and scale bake in wrong. Also run Ctrl+A → All Transforms first.
Apply Modifiers	On	Roblox does not run your modifier stack. You get the un-subdivided cage.
Object origin	At the object's own centre	The chunk system positions pieces by their centre. Origins left at the world origin make every piece stack in one spot.
Triangle count	10,000 max per MeshPart	The importer will offer to split it. Accept, then group the result.

Always check scale first, on one test import

The Ethereal Scape scene carries an **R6 character rig** in its `Scale_Reference` collection, and it is the most useful object in the file.

A Roblox R6 character is **5 studs tall, 2 wide, 1 deep**. Import the whole scene once, select that rig, and read its Size in Studio:

~5 studs tall → correct, carry on. ~0.5 → a 0.1× scale was applied. ~50 → a 10×.

Do this before splitting a scene into pieces. Getting it wrong afterwards means re-uploading every piece.

Uploading a mesh

- 1 In Studio: **Avatar tab** → **3D Importer**, choose your .fbx.
- 2 **Set the Creator correctly** if the experience is group-owned. An asset uploaded to a personal account **will not load** in a group experience — and it fails silently, as an invisible mesh rather than an error.
- 3 Select the imported MeshPart. **Properties** → **MeshId** reads `rbxassetid://1234567890`. That number is the deliverable.
- 4 Send us the ID with the logical name it belongs to. We paste it into one manifest file and the game picks it up — no other change anywhere.

You do NOT need to publish the place to upload a mesh

The 3D Importer uploads to your Roblox account from a local place file. Publishing the place is only needed for save data.

Building a prefab we can animate

This is the important section for the Fate Engine.

The Fate Engine has to react when a player rolls: rings spin up, the portal brightens, parts move, colours change to match what was rolled. All of that is done in code, at runtime — but code can only move a part that **exists as its own part**.

So: one part per moving piece

What you send	What we can do with it
The whole Engine as one mesh	Spin or bob the entire object. Tint it. Fade it. Nothing internal can move relative to anything else.
An .rbxmx with the outer ring, inner ring, plinth, runes and portal surface as separate parts inside one Model	Everything in the build spec: counter-rotating rings, a 2.5-second spin-up on roll, per-rarity colour, brightening lights, particle bursts, runes lighting in sequence.

Naming and pivots

- **Name every moving part clearly and consistently** — `OuterRing`, `InnerRing`, `Plinth`, `PortalPlane`. Code finds parts by name. A rename breaks the animation.

- **Set each part's pivot where it should rotate.** A ring that spins must have its pivot at the ring's centre, not at the model's origin — otherwise it orbits the whole build instead of turning in place.
- **Send a list of the part names** with the file. One line each: what it is, and how it should move. We wire it from that.
- **Add Attachments in Studio** where particles or lights should emit, before saving the file. Attachments do not survive an FBX export, but they do survive a Roblox model file.

Everything else is already handled

Anchoring, collision, lighting setup, the roll prompt, particle rates and the spin-up timing are all in code already, and are driven by settings rather than hard-coded. **You do not need to script anything.** Build the geometry, name the parts, place the attachments, save as .rbxm.

Saving an .rbxm

- 1 Assemble the parts under a single **Model** in Studio.
- 2 Set the Model's **PrimaryPart** — usually the base or plinth. This is the point we position the whole thing from.
- 3 Right-click the Model → **Save to File...** → choose **Roblox XML model files (*.rbxm)** → save into `assets/rbxm/` in the repo.
- 4 Commit it, or send us the file. Even as XML, git cannot sensibly merge two people's edits to the same prefab — only one person should edit a given file at a time.

A note on the folder being empty

`assets/rbxm/` is deliberately **not yet connected** to the sync configuration. An empty folder mapping breaks Rojo's sync, so the connection is made when the first prefab actually lands. That is a one-time change on our side — nothing for you to do, but it explains why the folder currently looks unused.

Working alongside the code

Do	Don't
Build in your own place file , save prefabs out as .rbxm.	Build directly in the shared live place.
Send mesh asset IDs for rigid props.	Edit any Script or ModuleScript in Studio. Rojo overwrites them and the change never reaches git.
Keep .blend sources in <code>assets/source/</code> .	Run a second Rojo server. Only one person runs it.
Tell us the part names and how each should move.	Name a model Crossroads . See the warning below.

One name is reserved: Crossroads

The hub currently builds itself from code every time the game starts. It checks for an existing model called **Crossroads** first, and if it finds one it **skips building entirely** and uses yours.

That is intended eventually — authored art should win. But it is **all or nothing**, and it happens **silently**. A model with that name would also switch off the spawn point, the roll prompt and the expedition gate, because those are built by the same step.

So: do not name anything Crossroads until we agree to make the switch together. Until then we swap in art **one piece at a time**, which needs no flag day and keeps the game playable throughout.

What happens after you send something

- 1 We add the asset ID or prefab path to one manifest file in the repo.
- 2 The game picks it up on the next start. Until then it draws a simple placeholder block in exactly the same position and size — so **a half-finished art pass is always playable**.
- 3 We wire up movement from your part-name list and test it.
- 4 If something needs changing, you re-export and we swap the ID. One line.

Quick checklist before you send

- Transforms applied (**Ctrl+A → All Transforms**)
- 1 metre = 1 stud, verified against the R6 rig
- Y up, -Z forward
- Each object's origin at its own centre
- Under 10,000 triangles per mesh, or split
- Moving pieces are separate parts, each with its pivot where it turns
- Parts named clearly; a list of names and intended movement included
- Attachments placed for any particles or lights
- Model has a PrimaryPart
- Nothing named **Crossroads**

Questions on any of this — ask before exporting a batch. A settings mistake is cheap to fix on one test object and expensive to fix on thirty.

Spec sheet: the Fate Engine and the Expedition Gate

If you are generating these in Blender — by hand or with an AI tool — build to these numbers and these names. They are read straight out of the game's config, so a model built to them drops in without anyone rescaling anything.

Who does what

A Blender tool produces a **.blend / .fbx**. It **cannot** produce a **.rbxmx** — that needs Roblox Studio.

So the flow is: **generate in Blender** → **export FBX** → **import to Studio** → **assemble into a Model** → **Save to File as .rbxmx**. The last two steps are a human in Studio, and they are where the part names and pivots get confirmed.

Required part names — these are not suggestions

The animation code finds parts **by name**. A renamed part is a part that stops moving. Use exactly these, one object per name:

```
FateEngine/ (or ExpeditionGate/)
  Plinth          the base you stand on
  OuterRing       rotates slowly one way
  InnerRing       rotates faster, the other way
  PortalPlane     the disc inside the inner ring
  RuneRing/
    Rune1 .. Rune8  8 plates around the plinth edge
    Glyph1 .. Glyph8 the lit inlay on each plate
```

Blender will happily name things `Cone.001`, `Circle.043` and so on. Rename before exporting, or the Studio step becomes an archaeology exercise.

Dimensions, in studs (1 Blender metre = 1 stud)

Piece	Fate Engine	Expedition Gate
Plinth	84 diameter × 3 tall	112 diameter × 3 tall
OuterRing	radius 54, section 7.2	radius 72, section 9.6
InnerRing	radius 39, section 3.6	radius 52, section 4.8
Ring centre height	57 above plinth base	75 above plinth base
PortalPlane	78 diameter disc	104 diameter disc
Rune plate ×8	13.2 × 1.8 × 20.4, at radius 33.6	17.6 × 2.4 × 27.2, at radius 44.8
Overall	84 wide, 111 tall	112 wide, 147 tall

The plinth is 3 studs tall and that is deliberate

A Roblox character jumps about 6.3 studs. An earlier version of this build scaled the plinth with everything else, reached 18 studs, and **walled the portal off completely** — nobody could get on it.

Width scales, height does not. Keep the plinth a **step**, not a wall.

Pivots

- **OuterRing and InnerRing:** pivot at the *ring's own centre*, on the ring axis. A pivot anywhere else makes the ring orbit the build instead of turning in place.
- **Plinth:** pivot at the centre of its *base*, not its middle — the whole rig is positioned from the floor up.
- **Everything else:** pivot at its own centre.

Orientation and geometry budget

- The rings stand **upright** — they are a doorway, not a halo. The ring plane is vertical; the portal disc fills the inner ring.
- Build it facing **-Z** in Blender (which becomes the Roblox facing after the Y up / -Z forward export).
- Rings are currently 28 straight segments. An authored torus can be smoother, but keep each ring **under 10,000 triangles** or it splits on import.
- **No text or lettering anywhere.** Generated glyphs come out as nonsense; the runes are geometric shapes only.

Materials

Send flat colours or simple materials, and expect to set the final look in Studio. FBX carries material *names*, not Blender's node graphs — so a procedural material arrives as an empty slot. Either bake to a texture, or name the slots clearly and we will assign Roblox materials to match. **Ring and portal colours are set at runtime anyway**, because they change to match what the player rolled.